ToolBridge — Enterprise MCP Gateway

The Complete Architecture, Intuitive Understanding & Practical Working Guide

DOCUMENT PURPOSE

Executive Summary: ToolBridge is an enterprise-grade Model Context Protocol (MCP) gateway and security firewall built in Python 3.12. It aggregates multiple upstream MCP servers (Filesystem, PostgreSQL, GitHub) under one collision-free namespace, protects internal assets with 7-layer filesystem jails and AST-level SQL inspection, enforces Role-Based Access Control (RBAC), and provides full operational visibility through interactive dashboards and an embedded AI copilot.

Part 1: The Intuitive Foundation — Why ToolBridge Exists

To understand ToolBridge, you first need to understand the problem it solves. Modern Large Language Models (LLMs) such as Claude, ChatGPT, and Cursor are brilliant at reasoning, but they cannot inherently interact with your private company databases, source code, or cloud APIs without external tools.

What is MCP (Model Context Protocol)?

Before MCP was created by Anthropic in late 2024, connecting an AI model to 10 different tools required 10 custom integrations. If you had 5 different AI clients, you had to maintain 50 custom integrations (the classic *N* × *M chaos*).

The Real-World Analogy: Think of MCP as **USB-C for Artificial Intelligence**. Just as USB-C replaced hundreds of proprietary charging cables with one universal plug, MCP standardizes how AI applications discover, inspect, and invoke external tools.

The Core Roles in the MCP Ecosystem:

- **MCP Client:** The application where the user talks to the AI (e.g., Claude Desktop, Cursor IDE, an autonomous agent).
- **MCP Server:** A program that exposes real-world capabilities (e.g., a Filesystem server or a PostgreSQL server).
- **MCP Tool:** A specific callable action declared by a server with a typed JSON schema (e.g., read_file or execute_query).

The Dangerous Reality of Direct Connections (Why an MCP Gateway is Required)

If MCP already standardizes tool connections, why can't companies simply let Claude Desktop connect directly to their PostgreSQL and Filesystem servers? In an enterprise, doing that is a severe security nightmare:

THE 5 FATAL PROBLEMS OF DIRECT MCP CONNECTIONS

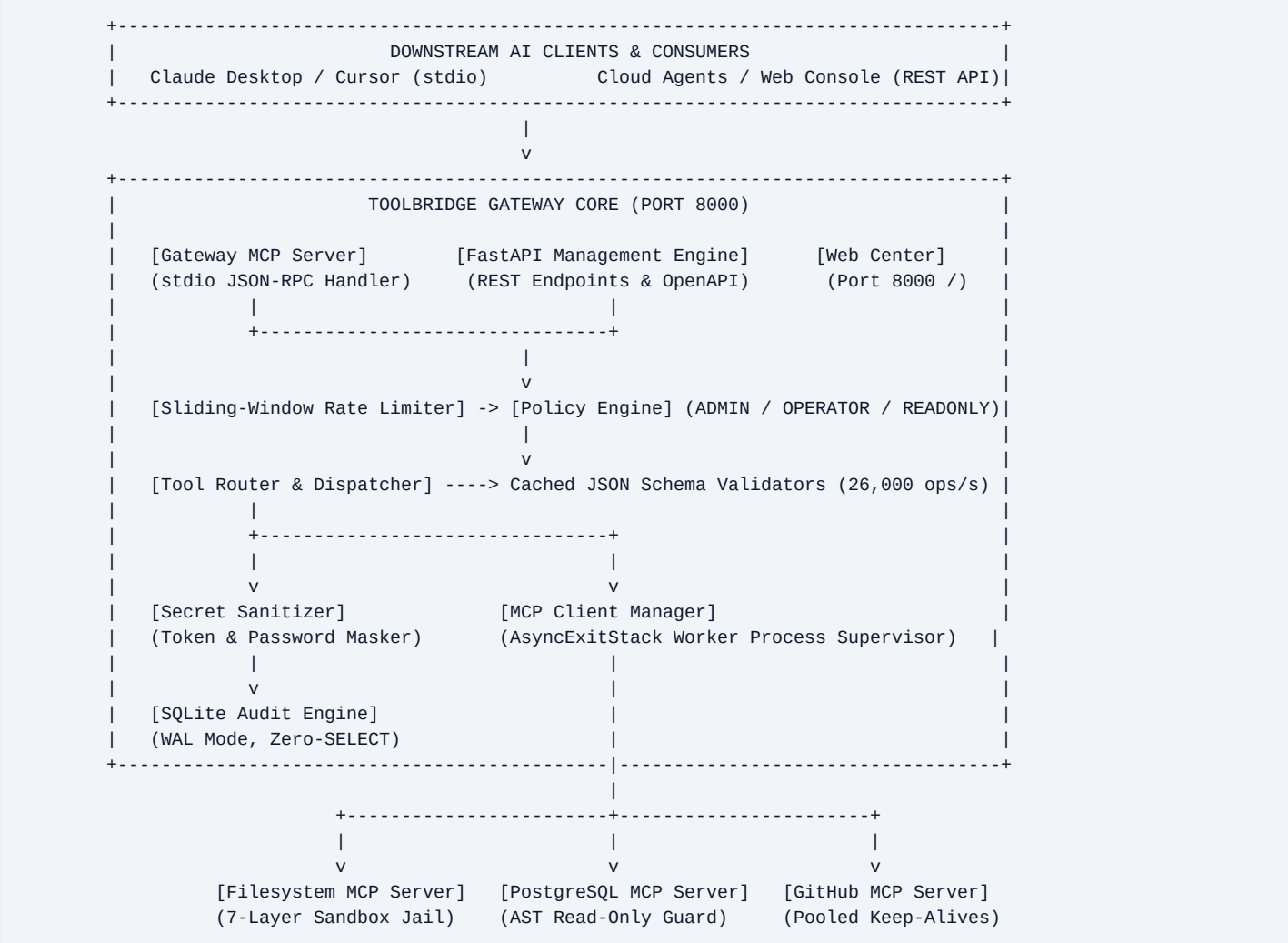
1. Zero Role-Based Access Control (RBAC): Standard MCP has no user permissions. A junior intern's desktop AI has the exact same destructive power as a database administrator.
2. Tool Name Collisions: If both GitHub and Jira servers declare a tool named search, the AI client crashes or invokes the wrong system.
3. Credential Exposure: Direct connections require putting production database passwords and GitHub tokens on every developer's local laptop.
4. Zero Centralized Audit Trail: If an AI hallucinates a command and wipes out data at 2:00 AM, there is no immutable log recording who initiated it.
5. No Protection Against Malicious AI Hallucinations: An AI model can easily be tricked by prompt injection into generating destructive SQL or escaping directories (e.g., ../../../../etc/passwd).

Enter ToolBridge: The Enterprise Security Firewall for AI

ToolBridge sits directly in the middle. It acts as a reverse proxy, traffic shaper, security filter, and centralized auditor. Downstream AI applications connect to *one single gateway*. ToolBridge securely controls all upstream systems, validates every argument, redacts every secret, and enforces strict authorization before any tool runs.

Part 2: Complete System Architecture (Component by Component)

ToolBridge is constructed using an asynchronous, modular architecture in Python 3.12. Below is the structural layout explaining how each component functions and communicates.



Detailed Component Breakdown:

1. The Front Door: Gateway MCP Server (stdio Transport)

• **Location:** src/toolbridge/server/gateway.py
• **What it does:** Implements the official MCP SDK (mcp.server.lowlevel.Server) listening on standard input/output. It acts as the single endpoint that Claude Desktop, Cursor, or local developer scripts connect to.
• **Why it matters:** To the AI client, ToolBridge looks like a single, comprehensive MCP server exposing dozens of tools.

2. The Management Plane: FastAPI API (HTTP REST)

• **Location:** src/toolbridge/api/app.py and routes/
• **What it does:** Operates on port 8000 to serve remote autonomous agents, microservices, and management dashboards. It enforces cryptographic API key authentication (32+ character keys with hmac.compare_digest constant-time comparison).

3. The Traffic Cop: Tool Router & Dispatcher

• **Location:** src/toolbridge/router/dispatcher.py
• **What it does:** Takes an incoming tool call (e.g., filesystem__read_file), inspects the namespace prefix, validates input parameters against pre-compiled JSON schemas (achieving 26,000+ validations per second), and proxies the request to the right upstream.
• **Why it matters:** Prevents invalid, malformed, or malicious payloads from ever reaching upstream servers.

4. The Security Shield: Policy Engine & RBAC

• **Location:** src/toolbridge/security/policy.py
• **What it does:** Classifies every tool into sensitivity tiers: **SAFE** (read-only queries, directory listing) and **SENSITIVE** (file writing, state changes). Enforces three distinct roles:
  • **ADMIN:** Unrestricted execution across all tools and server configuration.
  • **OPERATOR:** Permitted for safe queries and approved sensitive tools.
  • **READONLY:** Strictly confined to SAFE tools. Destructive attempts trigger an immediate HTTP 403 Forbidden.

5. The Upstream Supervisor: MCP Client Manager

• **Location:** src/toolbridge/client/manager.py
• **What it does:** Launches each upstream server as an isolated child OS process using AsyncExitStack. It monitors process health, handles cancellation notifications on timeouts, and recycles hung workers without crashing the gateway.

6. The Dedicated Upstream Guards

• **7-Layer Filesystem Jail (guards/filesystem.py):** Restricts file operations strictly inside ./workspace_sandbox. Neutralizes directory traversal (..), symlink escapes, Windows reserved device names (CON, PRN, AUX), and alternate data streams (::$DATA).
• **PostgreSQL AST Query Guard (guards/sql.py):** Uses sqlparse to tokenize queries. Strictly permits single-statement SELECT or EXPLAIN queries. Blocks mutations (INSERT/UPDATE/DELETE), stacked injection (SELECT 1; DROP TABLE users;), and administrative functions (pg_sleep, set_config, dblink).
• **GitHub REST Proxy:** Pools HTTP connections with keep-alives and transparent token management.

7. The Telemetry Engine: SQLite Audit Store & Secret Sanitizer

• **Location:** security/audit_service.py and security/sanitizer.py
• **What it does:** Before any record is written to disk, SecretSanitizer redacts passwords, Bearer tokens, GitHub PATs, and AWS keys. Records are committed asynchronously to SQLite running in WAL mode with composite indexes for sub-millisecond querying.

8. The Embedded AI Copilot: ToolBridge Assistant

• **Location:** src/toolbridge/assistant/ and Dashboard Tab 7
• **What it does:** An evidence-grounded AI copilot that helps operators diagnose 403 errors, simulate permissions for prospective roles (Phase B), and review human-approved action proposal cards (Phase C) with physical confirmation checkboxes.

Part 3: The Complete Working Process — How It Actually Works

To truly master ToolBridge, you must know what happens at every stage of execution. Below are the step-by-step lifecycles of five real-world scenarios.

Scenario 1: A Successful Safe Tool Request (e.g., Reading a File)

Step 1 — Ingestion: A developer in Cursor or an agent asks: *'Read notes.txt'*. The client sends a JSON-RPC request for filesystem__read_file with {'path': 'notes.txt'}.
Step 2 — Rate Limiting: The rate limiter checks the caller's ID in the $O(1)$ deque sliding window. Under 60 req/min? Approved.
Step 3 — Identity & Authorization: The Policy Engine checks the caller's role. filesystem__read_file is classified as SAFE. Whether the user is READONLY, OPERATOR, or ADMIN, the request is authorized.
Step 4 — Schema Validation: The pre-compiled JSON schema validator inspects the arguments. Path is a string? Schema checks pass in 0.03ms.
Step 5 — Upstream Execution: The client manager passes the call to the Filesystem MCP worker subprocess. The 7-layer jail verifies that notes.txt resolves safely within ./workspace_sandbox without escaping. The file is read non-blockingly.
Step 6 — Audit Logging & Response: The result is sanitized, the audit record is committed to SQLite (duration ~1.2ms), and the file content is returned to the AI.

Scenario 2: An Unauthorized Attempt Blocked by RBAC

Step 1 — Ingestion: An autonomous agent authenticated with a READONLY API key attempts to call filesystem__write_file.
Step 2 — Policy Evaluation: The Policy Engine looks up filesystem__write_file. It is classified as **SENSITIVE**. The policy rule states: *'READONLY callers are restricted strictly to SAFE tools'*.
Step 3 — Rejection: Execution halts immediately. The upstream server is never touched.
Step 4 — Audit & HTTP 403: An audit event is logged with status UNAUTHORIZED and reason *'Role readonly is not permitted to execute sensitive tool'*. The client receives an immediate HTTP 403 Forbidden.

Scenario 3: A Malicious SQL Injection Blocked by the AST Guard

Step 1 — Ingestion: An attacker attempts a stacked query through postgres__execute_query: SELECT * FROM users; DROP TABLE logs;.
Step 2 — AST Parsing: The query enters SQLSecurityGuard.validate_query(). sqlparse splits the query into a tokenized syntax tree.
Step 3 — Multi-Statement Detection: The guard counts statements. Two statements are detected. The guard immediately raises SQLSecurityError: Multi-statement execution is strictly prohibited. Exactly one query is permitted.
Step 4 — Database Safety: PostgreSQL never receives the malicious query. The database remains completely intact.

Scenario 4: Upstream Process Crash & Resilient Fallback

What happens if PostgreSQL daemon is not running on port 5432?
In traditional systems, the entire gateway crashes. In ToolBridge, the ResilientPostgresBackend checks connection health once. If port 5432 is unreachable, it logs a clear diagnostic to stderr and switches to an embedded SQLite sample database (sample_postgres.db). Subsequent queries execute in <0.7ms without repeating network timeouts. When upstream processes crash mid-call, the manager traps the EOF error, marks the server DEGRADED, and returns a clean error frame to the client rather than hanging.

Scenario 5: The ToolBridge Assistant Copilot Workflow

Step 1 — Operator Question: An operator opens Dashboard Tab 7 and asks: *'Why did my write_file call fail?'*
Step 2 — Grounding Assembly: The context builder fetches active tools, policy rules, and the operator's recent failure logs. All data is sanitized of passwords and tokens and wrapped in XML boundaries.
Step 3 — Model Reasoning: The model answers accurately with citations: [source: policy:filesystem__write_file].
Step 4 — Human-Approved Actions (Phase C): If the operator asks to reconnect a server, the assistant displays a **Proposed Action Card** with a physical checkbox: *'I authorize execution'*. Only upon administrator button click does the gateway execute the action.

Part 4: Practical Operator & Developer Guide

This section explains how to launch, configure, test, and demonstrate ToolBridge in minutes.

1. One-Command Quick Start

ToolBridge includes automated provisioning scripts for zero-friction startup. Simply run the launcher from the project root:

```
python run_demo.py
# Or on Windows:
.\demo.bat
```

What this command automatically does:

1. Generates cryptographically secure 32+ character API keys if not already present.
2. Initializes the SQLite database (./data/toolbridge.db) with WAL pragmas.
3. Seeds realistic audit records and security traces.
4. Starts the FastAPI Management Service at http://127.0.0.1:8000.
5. Launches the interactive Streamlit Control Plane at http://localhost:8501.

2. Accessing the Web Control Planes

• **FastAPI Web Control Center (http://127.0.0.1:8000/):** A lightweight single-page application served directly from FastAPI with 1-click test scenarios (safe queries, RBAC rejections, and SQL AST injection blocks).
• **Streamlit Enterprise Dashboard (http://localhost:8501):** Full administrative control plane featuring 7 specialized tabs:

1. **Gateway Overview:** Live metrics, invocation counts, success rates.
2. **Upstream Servers:** Process health, transport modes, tool counts.
3. **Tool Explorer & Runner:** Interactive playground to simulate roles.
4. **Audit Trail:** Real-time searchable log viewer with secret redaction indicators.
5. **Telemetry & Analytics:** Latency histograms and usage charts.
6. **Security Guardrails:** Visual inspection of filesystem, SQL, and rate limiter rules.
7. **Gateway Assistant:** Embedded conversational copilot and action approval cards.

3. Connecting Claude Desktop / Cursor

To connect Claude Desktop to ToolBridge as your unified downstream client, add this entry to your claude_desktop_config.json file:

```
{
  "mcpServers": {
    "toolbridge": {
      "command": "python",
      "args": ["-m", "toolbridge.server.gateway"],
      "env": {
        "TOOLBRIDGE_STDIO_ROLE": "operator"
      }
    }
  }
}
```

4. Executing Tools via REST API (curl)

External agents and microservices can execute tools through the management endpoint:

```
curl -X POST "http://127.0.0.1:8000/api/v1/tools/invoke" \
     -H "X-API-Key: YOUR_OPERATOR_KEY" \
     -H "Content-Type: application/json" \
     -d '{
       "tool_name": "filesystem__list_directory",
       "arguments": {"path": ""}
     }'
```

5. Running Automated Tests & Performance Benchmarks

Run the complete test suite (195 tests, 100% green):

```
.\.venv\Scripts\pytest -v
```

Run the empirical performance benchmark suite:

```
.\.venv\Scripts\python scripts/benchmark_performance.py
```

EMPIRICAL PERFORMANCE BENCHMARK RESULTS

Benchmark Highlights:

- **Rate Limiter:** 845,000+ checks/sec ($O(1)$ deque sliding window, P50 < 0.001 ms).
- **Secret Sanitizer:** 2,237,000+ ops/sec on benign payloads via fast-path heuristic.
- **Schema Validation:** 26,900+ dispatches/sec via pre-compiled validator cache.
- **Postgres Fallback:** 1,560+ queries/sec (<0.64 ms avg latency) with state-guarded fallback.
- **Filesystem Concurrency:** 100 parallel async operations isolated to non-blocking worker threads.

Part 5: Interview Defense Cheat Sheet

When explaining ToolBridge to an interviewer or engineering director, use these crisp, senior-level framing points.

Question: What is ToolBridge in one sentence?
'ToolBridge is to MCP what Kong or Apigee is to REST APIs—an enterprise gateway that sits between AI models and backend tools to provide centralized routing, role-based access control, AST SQL injection defense, and compliance auditing.'

Question: Why can't an AI app connect directly to MCP servers?
'Because direct connections have zero authorization controls, expose database passwords on laptops, risk prompt-injected SQL wipes or directory escapes, and leave no central audit trail. ToolBridge establishes the governance boundary required for production enterprise adoption.'

Question: How did you optimize its performance?
'We eliminated a 250ms TCP preflight timeout in fallback mode using connection state guards (412x speedup), pre-compiled and cached JSON schema validators (89.7x speedup), offloaded blocking disk I/O to worker threads via asyncio.to_thread, replaced rate limiter list-copies with O(1) deques, and removed redundant SELECT refresh queries from the SQLite audit commit path.'

Question: How does your ToolBridge Assistant avoid hallucinating or executing dangerous actions?
'Phase A is strictly read-only with zero tool-execution bindings. Context is deterministically assembled from the live registry and policy engine with XML-delimited prompt injection containment. For Phase C actionable requests, the assistant only generates a proposal card; execution requires an explicit physical checkbox and admin key confirmation.'

Repository: https://github.com/Bhaveshbargat1/ToolBridge--MCP-Gateway
Stack: Python 3.12 • MCP SDK v2.3.0 • FastAPI • Streamlit • SQLModel/SQLite • ReportLab
Verification Status: 195/195 Tests Passing (100% Green, 0 Skipped)